1.Insertion at Beginning

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again.
Learn with interactive visuals
Try Now
Programiz PRO ›

main.c Share Run Output Clear

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third,*newnode;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    newnode = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;
    newnode->data = 5;
```

```
List: 5 10 20 30

=== Code Execution Successful ===
```

C Online Compiler

main.c

```c

    first->data=223;
    first->next=second;

    second->data=300;
    second->next=NULL;

struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
newNode->data=98;
newNode->next=head;
head=newNode;


    struct Node*temp=head;
    while(temp!=NULL){
        printf("%d\n", temp-> data);
        temp = temp->next;
    }
    free (second);
    free (first);
    free (head);
    return 0;
    }
```

Output

```
98
106
223
300


=== Code Execution Successful ===
```

main.c

```c
    third->next = NULL;

    newnode->prev = NULL;
    newnode->next = head;
    head->prev = newnode;
    head = newnode;
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    printf("List: ");
    display();
    return 0;
}
```

Output

```
List: 5 10 20 30

=== Code Execution Successful ===
```

2. Insertion at End

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;
```

Output Clear

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

Share Run

```c
    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void insertend(){
    struct node *temp,*newnode;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 40;

    temp = head;

    while(temp->next != NULL){
        temp = temp->next;
    }
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

```c
    temp->next = newnode;
    newnode->prev = temp;
    newnode->next = NULL;
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertend();

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 20 30 40

=== Code Execution Successful ===
```

3. Insertion at Position

main.c

Share Run

```c
    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void insertpos(int pos){
    struct node *temp,*newnode;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 15;

    temp = head;

    for(i=1;i<pos-1;i++){
        temp = temp->next;
    }
```

Output

```
List: 10 15 20 30

=== Code Execution Successful
```

main.c

Share Run

```c
    newnode->next = temp->next;
    newnode->prev = temp;

    temp->next->prev = newnode;
    temp->next = newnode;
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertpos(2);

    printf("List: ");
    display();
```

Output

```
List: 10 15 20 30

=== Code Execution Successful ===
```

```
    printf("List: ");
    display();

    return 0;
}
```

4. Deletion at Beginning

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;
```

Output

```
List: 20 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void deletebegin(){
    struct node *temp;

    temp = head;
    head = head->next;
    head->prev = NULL;
    free(temp);
}

void display(){
    struct node *temp = head;
```

Output

```
List: 20 30

=== Code Execution Successful
```

main.c

```c
    head = head->next;
    head->prev = NULL;
    free(temp);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    deletebegin();

    printf("List: ");
    display();

    return 0;
}
```

Output

```
List: 20 30

=== Code Execution Successful
```

5. Deletion at End

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void create()
{
    struct node *first, *second, *third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

Share Run

```c
    head = first;

    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void deleteend()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c
    temp = head;

    while(temp->next != NULL)
    {
        temp = temp->next;
    }

    if(temp->prev == NULL)
    {
        head = NULL;
    }
    else
    {
        temp->prev->next = NULL;
    }

    free(temp);
}

void display()
{
    struct node *temp = head;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    create();

    printf("Before deletion:\n");
    display();

    deleteend();

    printf("\nAfter deletion:\n");
    display();

    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful =
```

6. Deletion at position

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void deletepos(int pos)
{
    struct node *temp;
    int i;

    temp = head;

    for(i = 1; i < pos; i++)
    {
        temp = temp->next;
    }
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
    if(temp->prev == NULL)   // deleting first node
    {
        head = temp->next;
        if(head != NULL)
            head->prev = NULL;
    }
    else if(temp->next == NULL)   // deleting last node
    {
        temp->prev->next = NULL;
    }
    else   // deleting middle node
    {
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    }

    free(temp);
}

void display()
{
    struct node *temp = head;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    struct node *first, *second, *third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;
```

Output


```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
    third->data = 30;

    head = first;

    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;

    printf("Before deletion:\n");
    display();

    deletepos(2);

    printf("\nAfter deletion:\n");
    display();

    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successf
```